

Smart Budget Tracker

Product Requirements Document

Version 1

Prepared by: Team Beta

Date: July 8, 2026

1. Overview & Problem Statement

Smart Budget Tracker is a personal finance web application that helps individuals log income and expenses, understand their spending patterns, and stay within self-defined budgets. Most people in our target market track finances inconsistently via memory, bank SMS alerts, or scattered notes with no unified view of where money goes each month.

This PRD defines the product's scope, functional requirements, the data model, metrics, and dashboard insights the app must deliver.

2. Goals & Success Metrics

Product Goals

- Let users log income/expense transactions quickly (under 15 seconds per entry).
- Give users a clear, at-a-glance picture of their financial health via a dashboard.
- Help users set and track budgets per category, with visibility into overspending.
- Surface actionable insights (trends, top spend categories, savings rate) without requiring manual analysis.

Success Metrics (v1)

Metric	Target	Notes
Weekly active users logging ≥ 1 transaction	$\geq 60\%$	Indicates habitual use
Avg. time to log a transaction	< 15 sec	UX/friction indicator
Users with ≥ 1 budget set	$\geq 50\%$	Adoption of budgeting feature
Dashboard session time	> 30 sec	Signals users are reading insights, not just entering data

3. Target Users

Persona	Description	Key Need
Budget-conscious professional	Salaried individual managing rent, transport, savings goals	Simple tracking + budget alerts
Side-hustler / freelancer	Irregular income streams (gigs, freelance)	Income vs. expense clarity across variable cash flow
Student / early-career	Limited, fixed income	Category-level spend visibility to avoid overspending

4. Competitive Analysis

Reviewed against four comparable apps to identify gaps we can exploit, particularly for the Nigerian market.

Feature	The Budgeting App	Goodbudget	Expenseure	PocketGuard
Core method	Manual log, pay-cycle based	Envelope budgeting	Manual + Nigerian bank sync	Automated bank sync, “safe to spend”
Bank sync	No	No	Yes (Nigerian banks)	Yes (~18k banks, Plaid/Finicity)
Standout feature	Pay-cycle flexibility, in-app calculator	Envelope system for family budgeting	Daily overspend alerts	“In My Pocket” safe-to-spend figure
Known weakness	No negative-balance rollover	Manual-entry heavy, dated UI	Nigeria-only relevance	Weak customization, basic charts

How Smart Budget Tracker Exploits These Gaps

- “Safe to Spend Today” metric — matches PocketGuard's core value but is cheap to add since we already capture income/expense/budget data: $(\text{Income} - \text{Fixed Bills} - \text{Budget Allocations} - \text{Savings Goal}) \div \text{Days Left in Period}$.
- Pay-cycle flexibility — users choose weekly/biweekly/monthly cycles instead of forced calendar-month budgeting, addressing irregular Nigerian pay dates that The Budgeting App handles but Goodbudget/PocketGuard don't.
- Fixed vs. Flexible category split — envelope-style grouping (rent, subscriptions vs. food, transport) inspired by Goodbudget, mapped onto our existing Budget vs. Actual table.
- Local bank/wallet sync, phased — lightweight local integration (SMS/email parsing or quick-add for GTBank, Opay, Kuda) planned as a fast-follow, directly targeting the gap Expenseure proves matters in this market.
- Debt / negative-balance carryover — explicitly fixes The Budgeting App's flagged weakness so overspending in one period rolls into the next instead of resetting.
- Shared/household budgeting — scoped out of v1 but flagged as a near-term differentiator, matching Goodbudget and Expenseure.

5. Scope

In Scope (v1 / MVP)

- Manual transaction entry (income & expense) with category, subcategory, account, notes.
- Predefined + user-defined categories, grouped as Fixed vs. Flexible.
- Budget setting per category, with configurable pay-cycle (weekly/biweekly/monthly).
- Dashboard: income vs. expense, spend by category, trend over time, budget vs. actual, Safe-to-Spend Today.
- Single currency (NGN) for v1.
- Single-user accounts (no shared/family accounts in v1).
- Debt/negative-balance carryover between pay periods.
- Export functionality (PDF/Excel reports)

Out of Scope (v1)

- Bank account auto-sync / Open Banking integration (planned as phased local integration post-v1).
- Multi-currency support.
- Automated transaction categorization via ML.
- Shared/family budgeting (flagged as fast-follow).
- Investment portfolio tracking.

6. Functional Requirements

6.1 Transaction Management

- Users can add, edit, delete income and expense transactions.
- Each transaction: date, type, category, subcategory, account, amount, payment method, notes.
- Transaction threshold(s) trigger — alert fixed at 90%/100% but user-configurable.
- Support recurring transactions (e.g., rent, salary) with auto-generation on schedule (optional by user).
- Bulk import via CSV (stretch goal for v1).

6.2 Categories & Budgets

- Default category list provided; users can add custom categories/subcategories.
- Categories grouped as Fixed (rent, subscriptions) or Flexible (food, transport).
- Users set a budget per expense category, tied to their chosen pay-cycle.
- System flags when spend in a category crosses a configurable threshold (e.g., 90%, 100%).

6.3 Dashboard & Insights

- Summary cards: total income, total expense, net savings, savings rate, Safe-to-Spend Today — for a selected period.
- Income vs. expense trend chart (weekly/monthly).
- Spend-by-category breakdown (pie/bar), split by Fixed vs. Flexible.
- Budget vs. actual table with variance and visual indicator (over/under budget).
- Date-range filter (this month, last 3 months, custom range).

6.4 Accounts

- Users can define multiple accounts (cash, bank, wallet) and assign transactions to them.
- Dashboard can filter by account.

7. Data Requirements (Data Analyst Ownership)

The core entities below define the data contract between frontend, backend, and analytics layers.

7.1 Entities & Key Fields

Entity	Key Fields
Transaction	id, user_id, date, type (income/expense), category_id, subcategory, account_id, amount, payment_method, notes, is_recurring
Category	id, name, type (income/expense), group (fixed/flexible), is_custom, parent_category_id (optional)
Account	id, user_id, name, type (cash/bank/wallet), currency
Budget	id, user_id, category_id, pay_cycle (weekly/biweekly/monthly), amount
User	id, name, email, currency_preference, pay_cycle_preference, created_at

7.2 Data Validation Rules

- Amount must be a positive number; sign is derived from transaction type, not entered by user.
- Date cannot be in the future beyond current day (except for scheduled recurring transactions).
- Every transaction must map to a valid category matching its type (income categories can't be used for expense transactions and vice versa).
- Duplicate detection: flag transactions with identical amount, date, and category within a short window for user review.

8. Key Metrics & Formulas

Metric	Formula
Total Income	SUM(amount) WHERE type = Income, within period
Total Expense	SUM(amount) WHERE type = Expense, within period
Net Savings	Total Income – Total Expense
Savings Rate	Net Savings ÷ Total Income
Budget Variance	Category Budget – Actual Spend
% Budget Used	Actual Spend ÷ Category Budget
Safe to Spend Today	(Income – Fixed Bills – Budget Allocations – Savings Goal) ÷ Days Left in Period

9. Non-Functional Requirements

- Data privacy: financial data is sensitive — encrypt at rest and in transit; no third-party data sharing.
- Performance: dashboard should load in under 2 seconds for up to 12 months of transaction history.
- Availability: 99.5% uptime target for v1.
- Auditability: all transaction edits/deletes should be logged (soft delete, not hard delete).

10. Assumptions & Constraints

- v1 targets a single currency (NGN) and single-user accounts only.
- No live bank integration in v1 — all entries are manual; local bank/wallet sync is a phased post-v1 goal.
- Mobile-responsive web app; native mobile app is a future phase, not v1.

11. Roles & Responsibilities

Each role below owns specific deliverables against this PRD.

Role	Name	Owens	Key Deliverables
Product Manager	Rufai Mubarak	Scope, prioritization, stakeholder alignment	Finalized PRD sign-off, roadmap, resolving open questions (Sec. 13)

Role	Name	Owens	Key Deliverables
Data Analyst	Uthman Toyib	Data model, metrics definitions, sample data, dashboard logic	Entity schema, KPI formulas, validation rules, sample dataset, Excel/Power BI mockup (done — see Phase 1)
UI/UX Designer	Ayoade Aliyu	Visual design, interaction flows	Wireframes, dashboard layout, design system, mobile-responsive views
Backend Engineer	Ajala Adeife	Database schema, API, business logic	Transactions/Categories/Budgets/Accounts tables, REST/GraphQL endpoints, recurring-transaction scheduler, validation enforcement
Frontend Engineer	Olaniyan Halimah	UI implementation, state management	Transaction entry forms, dashboard views, charts, filters, budget alerts UI
	Idris Abdulsalaam		
Cloud Engineer	Abdullateef Dauda	Hosting, infrastructure, deployment	Environment setup, CI/CD pipeline, database hosting, security/encryption config, uptime monitoring
QA / Tester	All team members	Validation & regression testing	Test cases for data validation rules, budget threshold alerts, dashboard accuracy

Suggested Workflow

- Data Analyst delivers the schema, sample dataset, and dashboard mockup (Phase 1 — complete).
- This feeds four functions in parallel: UI/UX Designer (wireframes and dashboard layout referencing the sample data), Backend Engineer (database schema and API build), Frontend Engineer (transaction entry forms and dashboard UI), and Cloud Engineer (hosting, environment setup, deployment pipeline).
- Open questions (Section 13) should be resolved by PM in a kickoff review before Phase 2 begins.

12. Milestones

Phase	Deliverable
Phase 1	Data model, sample dataset, dashboard mockup (Excel/Power BI) User flow diagrams + low-fi wireframes (entry pages, dashboard, budgets) Brand discovery workshop, mood boards, 3 logo concept directions
Phase 2	Backend schema + API for transactions/categories/budgets Finalize logo + deliver brand identity (colors, type, style guide) Develop logo animation + app intro motion concepts
Phase 3	Hi-fi dashboard mockups using draft brand direction Frontend: transaction entry + dashboard UI Deliver onboarding animation, loading states, transition assets
Phase 4	API hardening, bug fixes from QA UI polish, responsiveness fixes, accessibility pass Production deployment, uptime monitoring live, backup config Beta testing with sample users